

[Harvard CS50's Introduction to Cybersecurity

Lecture 3 – Securing Software (Personal Study Notes)

Disclaimer: These are my personal study notes created for learning and revision while taking Harvard CS50's Introduction to Cybersecurity. They are not official course materials.

1. Securing Software

Software security is the practice of designing and developing applications that can resist attacks and protect user data.

Every application receives input from users. If this input is not handled properly, attackers can exploit it to steal information, modify data, or gain unauthorized access.

Main Goal

- Protect user information.
 - Prevent unauthorized access.
 - Reduce software vulnerabilities.
 - Ensure applications behave only as intended.
-

2. Phishing

Definition

Phishing is a cyberattack where an attacker tricks people into revealing sensitive information such as usernames, passwords, bank details, or OTPs.

Instead of attacking the computer directly, phishing targets human trust.

Example

A fake website may look identical to a real banking website.

The displayed text may appear as:

<https://mybank.com>

But clicking it actually opens:

<https://fake-login.com>

Users who don't carefully verify the destination may unknowingly enter their credentials.

Prevention

- Always verify website URLs.
 - Hover over links before clicking.
 - Use Multi-Factor Authentication (MFA).
 - Avoid opening suspicious emails or messages.
-

3. Code Injection (Cross-Site Scripting – XSS)

Definition

Code Injection occurs when an application treats user input as executable code instead of plain text.

In Cross-Site Scripting (XSS), attackers inject JavaScript into a webpage. If the website fails to validate the input, the browser executes the malicious script.

Example

Instead of entering normal text into a search box, an attacker enters malicious JavaScript.

If the application displays this input without checking it, the browser executes the script.

Risks

- Session hijacking
- Cookie theft
- Fake login forms
- Redirecting users to malicious websites
- Displaying unwanted pop-up messages

Prevention

- Validate all user input.
 - Escape special characters.
 - Follow secure coding practices.
 - Implement Content Security Policies (CSP).
-

4. Reflected Attack

Definition

A Reflected XSS attack happens when malicious input is immediately returned by the website in its response without proper validation.

The harmful code is not stored on the server.

How it Works

1. Attacker creates a specially crafted URL.
2. Victim clicks the link.
3. The website reflects the malicious input.
4. The victim's browser executes the injected script.

Characteristics

- Temporary attack
 - Requires user interaction
 - Malicious code is reflected immediately
-

5. Stored Attack

Definition

A Stored XSS attack happens when malicious code is permanently stored by the website.

Whenever other users access the stored content, the malicious script automatically executes.

Example

An attacker posts malicious JavaScript inside a comment section.

Every visitor who opens that page unknowingly runs the script.

Risks

- Large number of victims
 - Persistent attack
 - Can steal cookies and user sessions
-

6. Character Escapes

Definition

Character Escaping is a security technique that converts special characters into harmless text before displaying them.

This prevents browsers from treating user input as executable code.

Why It Is Important

Some symbols have special meanings in HTML.

Examples include:

- <
- >
- &
- "
- '

If these characters are displayed directly, they may become part of executable HTML or JavaScript.

Escaping them ensures they appear only as normal text.

Common Character Escapes

Character Escaped Form

<	<
>	>
&	&
"	"
'	'

Benefits

- Prevents Cross-Site Scripting (XSS)
- Displays user input safely
- Stops browsers from executing malicious code
- Improves overall web application security

HTTP Headers :

What are HTTP Headers?

- HTTP headers are **extra information** sent between the **server** and the **browser**.
 - They tell the browser **how to handle and secure the webpage**.
-

Content Security Policy (CSP)

Definition:

Content Security Policy (CSP) is an HTTP security header that tells the browser **which sources are trusted** for loading resources such as JavaScript and CSS.

Example

```
Content-Security-Policy: script-src https://example.com/
```

Meaning:

- `script-src` → Rule for JavaScript.
- `https://example.com/` → Only JavaScript from `example.com` is allowed.

Browser Behavior

- ☐ Loads JavaScript from `https://example.com`.

- ☐ Blocks JavaScript from other websites or unauthorized inline scripts.
-

Who Sets and Who Enforces CSP?

- **Server (Website)** → Sends the CSP rules in the HTTP headers.
 - **Browser (User's browser)** → Reads the rules and enforces them by allowing or blocking resources.
-

Can an Attacker Bypass CSP?

- Attackers can **see the CSP rules**, but that does not let them bypass them.
 - CSP is **one layer of security**, not a complete defense.
-

Limitation of CSP

If the **trusted website (example.com)** itself is **compromised** and serves malicious JavaScript:

- The browser **will still execute it**, because it trusts the server's CSP policy.
 - **CSP cannot protect against a compromised trusted source.**
-

How Websites Protect Against This

Websites should also use:

- Secure coding practices.
 - Regular security updates and patching.
 - Code reviews and security testing.
 - Integrity checks (e.g., **Subresource Integrity (SRI)** for third-party scripts).
 - Strong server security and access controls.
-

QL Injection (SQL)

What is SQL?

Structured Query Language (SQL) is a language used to communicate with databases. It allows applications to store, retrieve, update, and delete data.

What is SQL Injection?

SQL Injection (SQLi) is a cyberattack in which an attacker inserts malicious SQL commands into user input. If the application directly includes this input in a database query without validation, the attacker may manipulate the database.

Example

A website searches for a user using the following query:

```
SELECT * FROM users
WHERE username = '{username}'
```

Here, the value entered by the user is directly inserted into the SQL query.

❑ **Security Rule:** Never trust user input. Always validate and sanitize every input before using it.

Example Attack 1 – Deleting Database Records

If an attacker enters:

```
malan'; DELETE FROM users; --
```

The final SQL query becomes:

```
SELECT * FROM users
WHERE username = 'malan'; DELETE FROM users; --'
```

Result

Instead of simply searching for a username, the database also executes the `DELETE FROM users` command.

This can remove every user account from the database.

Example Attack 2 – Bypassing Login Authentication

Consider the following login query:

```
SELECT * FROM users
WHERE username = '{username}'
AND password = '{password}'
```

Suppose the attacker enters the following as the password:

```
' OR '1'='1
```

The query becomes:

```
SELECT * FROM users
WHERE username = 'malan'
AND password = ''
OR '1'='1'
```

Since the condition `'1'='1'` is **always true**, the database may return every matching record instead of checking the correct password.

Risks of SQL Injection

- Unauthorized access to user accounts
 - Login authentication bypass
 - Viewing confidential information
 - Modifying database records
 - Deleting important data
 - Full database compromise
-

Prepared Statements

What are Prepared Statements?

Prepared statements are a secure way of writing SQL queries.

Instead of inserting user input directly into the SQL statement, placeholders are used and the database safely handles the input.

Example

Instead of writing:

```
SELECT * FROM users
WHERE username = '{username}'
```

Use:

```
SELECT * FROM users
WHERE username = ?
```


The user input is supplied separately rather than becoming part of the SQL command.

Why are they Secure?

Prepared statements automatically treat user input as **data**, not as SQL code.

Special characters such as single quotes are safely escaped, preventing injected SQL commands from executing.

Benefits

- Prevent SQL Injection attacks
 - Handle user input safely
 - Improve database security
 - Encourage secure coding practices
-

Command Injection

A command line interface is a method by which to run a computer system using text-based commands, as opposed to clicking on menus and buttons.

What is Command Injection?

Command Injection is an attack in which an attacker tricks an application into executing operating system commands.

This usually happens when user input is directly passed to the system command line without proper validation.

Common Cause

Applications that use functions such as:

- `system()`
- `eval()`

without validating user input can become vulnerable.

Example

Instead of entering normal input, an attacker may enter a malicious command that the operating system executes.

As a result, attackers may:

- Access sensitive files
- Delete important data
- Execute unauthorized programs
- Gain control over the system

Prevention

- Never trust user input.
- Validate and sanitize all inputs.
- Escape special characters before passing input to system commands.
- Prefer secure APIs instead of directly executing shell commands.
- Always follow the official documentation for securely handling user input.

Developer Tools

What are Developer Tools?

Developer Tools are built-in features available in modern web browsers that allow users to inspect and interact with the underlying HTML, CSS, and JavaScript of a webpage.

They help developers understand how a webpage works, debug issues, and test changes. However, they can also demonstrate why relying only on browser(client)-side security is risky.(Developer Tools demonstrate that client-side validation can be bypassed. Therefore, security checks must always be performed on the server as well.)

Inspecting HTML Elements

Developer Tools allow users to view and temporarily modify the HTML code of a webpage that is loaded in their own browser.

For example, consider the following HTML:

```
<input disabled type="checkbox">
```

Explanation

- `type="checkbox"` creates a checkbox.
- The `disabled` attribute makes the checkbox inactive.
- Because of the `disabled` attribute, the user cannot interact with the checkbox through the webpage.

Why is this a Security Concern?

The HTML of a webpage is downloaded to the user's browser.

Since the user has access to this local copy, they can inspect and modify it using Developer Tools.

For example, a user can remove the `disabled` attribute.

Original HTML:

```
<input disabled type="checkbox">
```

Modified HTML:

```
<input type="checkbox">
```

Result

After removing the `disabled` attribute, the checkbox becomes active and can now be selected.

This change affects **only the user's local browser**. The original HTML on the server remains unchanged.

Never Trust Client-Side Validation

Client-side validation refers to checks performed by the browser before sending data to the server.

Although it improves user experience by catching simple mistakes quickly, it **should never be considered a security mechanism**.

Example

Consider the following HTML:

```
<input required type="text">
```

Explanation

- `type="text"` creates a text input field.
 - The `required` attribute prevents the form from being submitted if the field is left empty.
-

Bypassing Client-Side Validation

Using Developer Tools, a user can simply remove the `required` attribute.

Original HTML:

```
<input required type="text">
```

Modified HTML:

```
<input type="text">
```

Result

The browser no longer requires the field to be filled before submitting the form.

This shows that browser-based validation can easily be bypassed by anyone who has access to Developer Tools.

Why is Server-Side Validation Important?

Since users can modify HTML in their own browsers, applications **must never rely only on client-side validation**.

The server should always verify every piece of user input before processing it.

Server-side validation helps ensure that:

- User input follows the expected format.
 - Invalid or malicious data is rejected.
 - Security checks cannot be bypassed by modifying browser-side code.
-

Key Points to Remember

- Developer Tools allow users to inspect and modify the HTML of a webpage in their own browser.
- HTML changes made through Developer Tools affect only the local copy, not the server.
- Attributes such as `disabled` and `required` can be removed by the user.
- Client-side validation improves usability but **does not provide complete security**. (Client-side validation **improves usability** by giving users **instant feedback** without waiting for the server. For example, if a required field is left empty, the browser immediately asks the user to fill it, making the form faster and more user-friendly. However, it **does not provide security** because users can bypass or modify these checks.)
- Sensitive validation must always be performed on the **server side**.
- **Golden Rule:** Never trust client-side validation alone. Always validate user input on the server.

Server-Side Validation

What is Server-Side Validation?

Server-side validation is the process of checking user input **on the server** before it is processed or stored.

Unlike client-side validation, users cannot bypass server-side validation by modifying the HTML or using Developer Tools.

Why is it Important?

Server-side validation helps ensure that:

- User input is valid.
- Malicious or unexpected data is rejected.
- Applications remain secure even if client-side validation is bypassed.

Important Principle

Never trust user input.

Every piece of data received from a user should be validated on the server before being accepted.

Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) is a cyberattack in which an attacker tricks a logged-in user's browser into sending an unwanted request to another trusted website without the user's knowledge or permission.

The attacker does **not** need the user's username or password. Instead, they take advantage of the user's active login session.

HTTP Methods Used in CSRF

Websites mainly communicate with users using two HTTP methods:

1. GET Method

The **GET** method is used to **request or retrieve information** from the server.

Example:

```
<a href="https://www.amazon.com/dp/B07XLQ2FSK">Buy Now</a>
```

When the user clicks the link, the browser sends a **GET request** to the server, and the server returns a response.

2. POST Method

The **POST** method is used to **send data** to the server.

It is commonly used for

- Login forms
- Registration forms
- Online purchases
- Updating account information

After processing the request, the server also sends a response back to the browser.

How Does a CSRF Attack Work?

Imagine a user is already logged into Amazon.

The browser stores the user's **session cookie**, which tells Amazon that the user is authenticated.

Now the user visits an attacker's website.

The attacker secretly includes the following code:

```
<form action="https://www.amazon.com/" method="post">
<input name="dp" type="hidden" value="B07XLQ2FSK">
<button type="submit">Buy Now</button>
</form>

<script>
document.forms[0].submit();
</script>
```

The JavaScript automatically submits the form without the user's knowledge.

Since the browser automatically sends the user's session cookie along with the request, Amazon believes that the request came from the legitimate user.

As a result, the unwanted action may be performed.

Why is it Called Cross-Site Request Forgery?

- **Cross-Site** → One website (attacker's website) causes the browser to send a request to another trusted website.
- **Request Forgery** → The request appears to come from the legitimate user even though it was triggered by the attacker.

Why is CSRF Dangerous?

A successful CSRF attack may cause the user to unknowingly:

- Purchase a product
- Transfer money
- Change account settings
- Update passwords
- Perform other sensitive actions

Preventing CSRF

One of the most common protection mechanisms is a **CSRF Token**.

What is a CSRF Token?

A CSRF Token is a unique secret value generated by the server for each user session.

Whenever a request is submitted, the server checks whether the received token matches the expected token.

- If the token is valid → The request is accepted.
- If the token is missing or invalid → The request is rejected.

Since attackers cannot predict or generate the correct token, they cannot successfully forge requests.

CSRF tokens are commonly included in HTML forms or HTTP headers.

NOTE:

- **The CSRF token is generated by the website's server (e.g., Amazon's server).**
- It is **sent to your browser** and is usually stored **inside the Amazon webpage** (for example, as a hidden form field or in page data), not as a general browser-wide value.
- The **server also keeps or can verify** the expected token for your session.

If you're logged into **Amazon**, **Google**, and **Facebook** at the same time:

- ☐ Amazon has its **own CSRF token**.
- ☐ Google has its **own CSRF token**.
- ☐ Facebook has its **own CSRF token**.

Each website manages its **own separate token** for your session. They are **not shared between websites**.

Key Points to Remember

- CSRF tricks a logged-in user's browser into sending unwanted requests to another trusted website.
- The attacker exploits the user's active session instead of stealing the user's password.
- GET requests mainly retrieve information, while POST requests mainly send data to the server.
- Both GET and POST requests receive responses from the server.
- Browsers automatically include session cookies with requests to trusted websites.
- CSRF Tokens help the server verify that a request genuinely originated from the authenticated user and protect against forged requests.

Arbitrary Code Execution (ACE)

What is Arbitrary Code Execution (ACE)?

Arbitrary Code Execution (ACE) is a security vulnerability in which a program executes code that **was never intended by the software developer**. Instead of running only the legitimate code, the application is manipulated into executing malicious code supplied by an attacker.

Such attacks may allow attackers to gain unauthorized access, modify the program's behavior, or compromise the entire system.

Threats that can Lead to Arbitrary Code Execution

1. Buffer Overflow

One common threat that can result in Arbitrary Code Execution is **Buffer Overflow**.

A buffer overflow occurs when a program receives **more input than the allocated memory can store**. The extra input spills into nearby memory locations, causing the software to behave unexpectedly.

For example, if an application expects only a short input but receives an extremely long input, the excess data may overwrite important memory and cause the application to malfunction.

In some cases, attackers can exploit this behavior to execute malicious code.

2. Stack Overflow

Another related threat is **Stack Overflow**.

A stack overflow occurs when data overflows into the program's stack memory. Attackers may exploit this vulnerability to insert and execute malicious code.

Like buffer overflows, stack overflows can also lead to Arbitrary Code Execution if proper memory protection is not implemented.

Other Uses of These Attacks

Buffer overflow and stack overflow attacks are not only used to execute malicious code.

Attackers may also use them to:

- Bypass software registration or licensing.
 - Crack paid software.
 - Reverse engineer applications to understand how they work internally.
-

Open-Source Software

What is Open-Source Software?

Open-source software is software whose **source code is publicly available** for anyone to inspect, study, and review.

Since the code is openly accessible, developers and security researchers can audit it to identify bugs and security vulnerabilities.

As a result, many security issues can be discovered and fixed by the community.

However, open-source software is not completely secure and can still be vulnerable to attacks.

Closed-Source Software

What is Closed-Source Software?

Closed-source software is software whose **source code is not publicly available**.

Only the software owner or organization has access to the source code.

Because the code is hidden from the public, it may be more difficult for attackers to study the software.

However, this also means that fewer people can inspect the code to identify security vulnerabilities.

Open-Source vs Closed-Source

There is a trade-off between open-source and closed-source software.

- **Open-source software** allows many developers and security experts to examine the code, making it easier to discover and fix vulnerabilities.
- **Closed-source software** hides the source code from the public, reducing direct visibility to attackers, but security mainly depends on the software vendor to identify and fix vulnerabilities.

There is **no universal winner**.

- **Linux** is generally considered more secure for servers, cybersecurity work, and developers. (open one)
- **Windows** is very secure today as well, especially with modern security features, but it is targeted more often because it has a much larger desktop user base. (closed one)

Easy one-line conclusion

Open source provides transparency, while closed source provides secrecy. Neither guarantees security—regular updates, secure design, and timely patching are what truly make software secure.

Key Points to Remember

- **Arbitrary Code Execution (ACE)** allows attackers to execute unintended code inside a program.
- **Buffer Overflow** and **Stack Overflow** are common threats that can lead to ACE.
- These attacks may also be used for software cracking and reverse engineering.
- **Open-source software** allows public code review but is still vulnerable to attacks.
- **Closed-source software** keeps the source code private but relies mainly on the vendor for security updates.

App Stores

What are App Stores?

App stores are platforms managed by organizations such as **Google** and **Apple**, where developers submit their applications for review before they are made available to users.

Before approving an application, the app store checks the software for any malicious or harmful behavior.

As a result, downloading applications from an official app store is generally much safer than installing software directly from unknown or untrusted developers.

How Do App Stores Improve Security?

App stores use **encryption** and **digital signatures** to verify that software comes from authorized developers.

Developers digitally sign their applications before submitting them to the app store.

The app store then verifies the application and signs it with its own digital signature.

When a user installs the application, the operating system checks these digital signatures to ensure that the software is authentic and has not been modified.

This helps prevent unauthorized or malicious software from being installed.

Package Managers

What are Package Managers?

Package managers are software tools that allow users to download, install, update, and manage software packages from trusted repositories.

Like app stores, package managers also use **digital signatures** to verify that downloaded software comes from a trusted source.

This helps users confirm that the software has not been altered before installation.

However, **digital signatures do not guarantee that software is completely secure.** Vulnerabilities may still exist.

The main goal is to make it much more difficult for attackers to distribute malicious software.

Bug Bounty

What is a Bug Bounty?

A bug bounty is a program in which organizations **reward security researchers or ethical hackers** for discovering and responsibly reporting security vulnerabilities in their software.

Instead of exploiting the vulnerabilities, researchers report them to the organization and receive a reward.

Bug bounty programs encourage responsible disclosure and help organizations improve their software security.

Identifying Vulnerabilities

Developers continuously monitor security databases to identify known vulnerabilities and understand their severity.

Several important resources are used for this purpose.

1. Common Vulnerabilities and Exposures (CVE)

A **Common Vulnerabilities and Exposures (CVE)** is a publicly identified and documented security vulnerability.

Each vulnerability is assigned a unique **CVE ID**, allowing developers and security professionals worldwide to refer to the same issue consistently.

Developers use CVE databases to stay informed about newly discovered vulnerabilities.

2. Common Vulnerability Scoring System (CVSS)

The **Common Vulnerability Scoring System (CVSS)** is used to measure the **severity** of a security vulnerability.

It assigns a numerical score based on factors such as:

- Ease of exploitation
- Potential impact
- Risk to affected systems

Higher scores indicate more severe vulnerabilities.

Developers use CVSS scores to determine which vulnerabilities should be addressed first.

3. Exploit Prediction Scoring System (EPSS)

The **Exploit Prediction Scoring System (EPSS)** estimates the **likelihood that a vulnerability will be exploited in real-world attacks**.

Unlike CVSS, which measures severity, EPSS predicts the probability of exploitation.

This helps organizations prioritize vulnerabilities that are most likely to be targeted by attackers.

4. Known Exploited Vulnerabilities (KEV)

The **Known Exploited Vulnerabilities (KEV)** database contains vulnerabilities that are **confirmed to have been actively exploited by attackers**.

Developers and organizations use the KEV database to quickly identify and patch vulnerabilities that are already being used in real-world cyberattacks.

Key Points to Remember

- **App Stores** review applications and use digital signatures to ensure software comes from authorized developers.
- Installing software from official app stores is generally safer than downloading it from unknown sources.

- **Package Managers** also verify software using digital signatures but cannot guarantee complete security.
- **Bug Bounty** programs reward ethical hackers for responsibly reporting security vulnerabilities.
- **CVE** provides unique identifiers for publicly known vulnerabilities.
- **CVSS** measures the severity of a vulnerability.
- **EPSS** predicts the likelihood that a vulnerability will be exploited.
- **KEV** lists vulnerabilities that are already known to be actively exploited.